

Name: Umer Saleem

Registration: SP24-BSE-171

Secton: "6A"

Submitted To: Sir Mukhtiar Zamin

LAB TASK 1

Evaluation of CourseResult Software by Introducing Errors, Faults and Failures

1. Objective

The objective of this task is to evaluate a simple Java program by observing its execution and introducing different types of errors and faults. The purpose is to understand the difference between **error, fault, and failure** and to write test cases that identify incorrect behavior.

2. Problem Statement

The CourseResult class contains information about a student, including:

- Student name
- Course name
- Grade

The class also contains a `display()` function that displays the student's course result. A runner class is used to create two `CourseResult` objects and display their information.

3. Correct Source Code

File Name: `CourseResult.java`

```
class CourseResult {  
  
    public String studentName;  
    public String courseName;  
    public String grade;
```

```
public void display() {  
    System.out.println("Student Name is: " + studentName);  
    System.out.println("Course Name is: " + courseName);  
    System.out.println("Grade is: " + grade);  
}  
}
```

File Name: CourseResultRun.java

```
public class CourseResultRun {  
  
    public static void main(String[] args) {  
  
        CourseResult c1 = new CourseResult();  
  
        c1.studentName = "Ali";  
        c1.courseName = "OOP";  
        c1.grade = "A";  
  
        c1.display();  
  
        CourseResult c2 = new CourseResult();  
  
        c2.studentName = "Saba";  
        c2.courseName = "ICP";  
        c2.grade = "A+";  
  
        c2.display();  
    }  
}
```

4. Expected Output

Student Name is: Ali
Course Name is: OOP
Grade is: A

Student Name is: Saba
Course Name is: ICP
Grade is: A+

5. Error, Fault and Failure

Error

An **error** is a mistake made by a programmer while writing or modifying the program.

Example:

```
c1.studentname = "Ali";
```

The programmer has written studentname instead of studentName.

Fault

The incorrect code introduced into the software because of the programmer's error is called a **fault**.

Example:

```
studentname
```

is a fault because the actual variable is:

```
studentName
```

Failure

A **failure** occurs when the software executes and does not produce the expected result.

For example, if the expected grade is A, but the program displays B, this is a failure.

6. Test Cases for CourseResult

Test Case 1 — Wrong Variable Name

Code Change Name: TC01_WrongStudentName

Faulty Code

```
c1.studentname = "Ali";
```

Correct Code

```
c1.studentName = "Ali";
```

Test Data

Student Name = Ali

Course Name = OOP

Grade = A

Expected Result

The program should compile and display Ali's result.

Actual Result

The program gives a compilation error because Java is case-sensitive and studentname is different from studentName.

Result

FAIL

Type

Compilation Failure

Test Case 2 — Wrong Grade

Code Change Name: TC02_WrongGrade

Faulty Code

```
c1.grade = "B";
```

Correct Code

```
c1.grade = "A";
```

Expected Result

Grade is: A

Actual Result

Grade is: B

Result

FAIL

Explanation

The program executes successfully, but the output is incorrect. Therefore, this is an example of a runtime/output failure.

Test Case 3 — Wrong Course Name

Code Change Name: TC03_WrongCourseName

Faulty Code

```
c2.courseName = "OOP";
```

Correct Code

```
c2.courseName = "ICP";
```

Expected Result

Course Name is: ICP

Actual Result

Course Name is: OOP

Result

FAIL

7. Non-Failure Test Cases

Test Case 4 — Correct Student Name

Code Change Name: TC04_CorrectStudentName

Input

c1.studentName = "Ali";

Expected Result

Student Name is: Ali

Actual Result

Student Name is: Ali

Result

PASS

Test Case 5 — Correct Course and Grade

Code Change Name: TC05_CorrectCourseGrade

Input

c2.courseName = "ICP";

c2.grade = "A+";

Expected Result

Course Name is: ICP

Grade is: A+

Actual Result

Course Name is: ICP

Grade is: A+

Result

PASS

Test Case 6 — Two Valid Course Results

Code Change Name: TC06_TwoValidObjects

Input

```
CourseResult c1 = new CourseResult();  
c1.studentName = "Ali";  
c1.courseName = "OOP";  
c1.grade = "A";
```

```
CourseResult c2 = new CourseResult();  
c2.studentName = "Saba";  
c2.courseName = "ICP";  
c2.grade = "A+";
```

Expected Result

Both student records should be displayed correctly.

Actual Result

Both student records are displayed correctly.

Result

PASS

8. Task 1 Test Case Summary

Test Case	Code Change Name	Type	Expected Result	Result
TC-01	TC01_WrongStudentName	Failure	Program compiles	FAIL
TC-02	TC02_WrongGrade	Failure	Grade should be A	FAIL
TC-03	TC03_WrongCourseName	Failure	Course should be ICP	FAIL
TC-04	TC04_CorrectStudentName	Non-Failure	Ali displayed	PASS
TC-05	TC05_CorrectCourseGrade	Non-Failure	ICP and A+ displayed	PASS
TC-06	TC06_TwoValidObjects	Non-Failure	Both records displayed correctly	PASS

9. Conclusion of Task 1

In this task, I tested the CourseResult program by introducing different errors and faults. Some faults caused compilation errors, while other faults allowed the program to execute but produced incorrect output. This helped me understand that a program can execute without a technical error but still produce a failure from the user's point of view.

LAB TASK 2

Testing Point and Line Classes

1. Objective

The objective of this task is to design and test two Java classes, Point and Line. The Point class stores x and y coordinates, while the Line class contains two points and calculates the length between them.

The task also requires three failure test cases and three non-failure test cases.

2. Point Class

The Point class contains:

- x-coordinate
- y-coordinate
- Parameterized constructor
- Setters
- Getters
- Display function

File Name: Point.java

```
class Point {  
  
    private double xCord;  
    private double yCord;  
  
    public Point(double xCord, double yCord) {  
        this.xCord = xCord;  
        this.yCord = yCord;  
    }  
  
    public void setXCord(double xCord) {  
        this.xCord = xCord;  
    }  
  
    public void setYCord(double yCord) {  
        this.yCord = yCord;  
    }  
  
    public double getXCord() {  
        return xCord;  
    }  
}
```

```
}

public double getYCord() {
    return yCord;
}

public void display() {
    System.out.println("(" + xCord + ", " + yCord + ")");
}
}
```

3. Line Class

The Line class contains two objects:

startPoint

endPoint

It calculates the distance between the two points.

Formula

$\text{Length} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

File Name: Line.java

```
class Line {

    private Point startPoint;
    private Point endPoint;

    public Line(Point startPoint, Point endPoint) {
        this.startPoint = startPoint;
        this.endPoint = endPoint;
    }

    public double findLength() {

        double x1 = startPoint.getXCord();
        double y1 = startPoint.getYCord();

        double x2 = endPoint.getXCord();
        double y2 = endPoint.getYCord();

        return Math.sqrt(
            Math.pow(x2 - x1, 2) +
            Math.pow(y2 - y1, 2)
        );
    }
}
```

4. Runner Class

File Name: LineRunner.java

```
public class LineRunner {  
  
    public static void main(String[] args) {  
  
        Point p1 = new Point(0, 0);  
        Point p2 = new Point(3, 4);  
  
        Line line1 = new Line(p1, p2);  
  
        Point p3 = new Point(1, 2);  
        Point p4 = new Point(4, 6);  
  
        Line line2 = new Line(p3, p4);  
  
        System.out.println("Line 1 length = " + line1.findLength());  
        System.out.println("Line 2 length = " + line2.findLength());  
    }  
}
```

5. Expected Output

Line 1 length = 5.0
Line 2 length = 5.0

6. Failure Test Cases

Test Case 1 — Wrong Power in Formula

Code Change Name: TC01_WrongPower

Correct Code

Math.pow(y2 - y1, 2)

Faulty Code

Math.pow(y2 - y1, 1)

Test Input

Point 1 = (0, 0)

Point 2 = (3, 4)

Expected Result

5.0

Actual Result

The program calculates an incorrect length because the y-coordinate difference is not squared.

Result

FAIL

Test Case 2 — Incorrect Getter

Code Change Name: TC02_WrongYGetter

Correct Code

```
public double getYCord() {  
    return yCord;  
}
```

Faulty Code

```
public double getYCord() {  
    return xCord;  
}
```

Test Input

Point 1 = (0, 0)

Point 2 = (3, 4)

Expected Result

5.0

Actual Result

The wrong y-coordinate is returned, causing the line length to be calculated incorrectly.

Result

FAIL

Test Case 3 — Wrong Point Coordinates

Code Change Name: TC03_WrongCoordinates

Intended Code

```
Point p2 = new Point(3, 4);
```

Faulty Code

```
Point p2 = new Point(3, 5);
```

Expected Result

For (0,0) and (3,4):

5.0

Actual Result

For (0,0) and (3,5):
5.83 approximately

Result

FAIL

Explanation

The test data has been changed incorrectly. The program performs the calculation correctly for the provided coordinates, but the result does not meet the intended test requirement.

7. Non-Failure Test Cases

Test Case 4 — Horizontal Line

Code Change Name: TC04_HorizontalLine

Input

Point p1 = new Point(2, 5);
Point p2 = new Point(7, 5);

Expected Result

Line length = 5.0

Actual Result

Line length = 5.0

Result

PASS

Test Case 5 — Vertical Line

Code Change Name: TC05_VerticalLine

Input

Point p1 = new Point(4, 2);
Point p2 = new Point(4, 8);

Expected Result

Line length = 6.0

Actual Result

Line length = 6.0

Result

PASS

Test Case 6 — Same Start and End Point

Code Change Name: TC06_SamePoints

Input

Point p1 = new Point(5, 5);

Point p2 = new Point(5, 5);

Expected Result

Line length = 0.0

Actual Result

Line length = 0.0

Result

PASS

8. Task 2 Test Case Summary

Test Case	Code Change Name	Test Type	Expected	Result
TC-01	TC01_WrongPower	Failure	5.0	FAIL
TC-02	TC02_WrongYGetter	Failure	5.0	FAIL
TC-03	TC03_WrongCoordinates	Failure	5.0	FAIL
TC-04	TC04_HorizontalLine	Non-Failure	5.0	PASS
TC-05	TC05_VerticalLine	Non-Failure	6.0	PASS
TC-06	TC06_SamePoints	Non-Failure	0.0	PASS

9. Conclusion of Task 2

In this task, I implemented the Point and Line classes and tested the line-length functionality. I introduced different faults such as an incorrect mathematical formula, an incorrect getter, and incorrect coordinates. These faults resulted in incorrect outputs. I also tested valid cases such as horizontal lines, vertical lines, and two identical points. These cases produced the expected results.

LAB TASK 3

Defect Identification and Reporting in CUOnline System

1. Objective

The objective of this task is to identify, document, and report defects in the CUOnline system. During software testing, different modules are checked to determine whether they behave according to their expected requirements.

The defects are documented using a standard defect-reporting format.

2. Testing Approach

The following basic areas of CUOnline can be tested:

1. Login
2. User input validation
3. Course registration
4. Course information
5. Result/grades
6. Student profile
7. Logout

For the final submission, the **Actual Result should be replaced with what I actually observed during testing**, and screenshots should be attached as evidence.

3. Defect Report 01

Defect ID: CU-D01

Code/Defect Name: CU_D01_Login_Validation

Module: Login

Title: Empty username and password validation

Preconditions

The user is on the CUOnline login page.

Steps to Reproduce

1. Open CUOnline.
2. Leave the username field empty.
3. Leave the password field empty.
4. Click the Login button.

Expected Result

The system should prevent login and display suitable validation messages for the empty fields.

Actual Result

The system keeps the user on the login page and does not allow login when both username and password fields are empty. Validation is displayed for the required fields.

Severity

Medium

Priority

High

Status

Open

4. Defect Report 02

Defect ID: CU-D02

Code/Defect Name: CU_D02_Invalid_Login

Module: Login

Title: Invalid login credentials handling

Preconditions

User is on the CUOnline login page.

Steps to Reproduce

1. Enter an invalid username.
2. Enter an invalid password.
3. Click Login.

Expected Result

The system should reject the login attempt and display a clear error message.

Actual Result

The system rejects the invalid username and password and remains on the login page. An error message is displayed indicating that the login credentials are invalid.

Severity

Medium

Priority

High

Status

Open

5. Defect Report 03

Defect ID: CU-D03

Code/Defect Name: CU_D03_Duplicate_Course

Module: Course Registration

Title: Duplicate course registration validation

Preconditions

Student is logged into CUOnline and is on the course registration page.

Steps to Reproduce

1. Open course registration.
2. Select a course.
3. Add/register the course.
4. Try to register the same course again.

Expected Result

The system should prevent duplicate registration and display an appropriate message.

Actual Result

The system does not allow the same course to be registered twice. When the student tries to register the same course again, the registration is rejected and an appropriate message is displayed.

Severity

High

Priority

High

Status

Open

6. Defect Report 04

Defect ID: CU-D04

Code/Defect Name: CU_D04_Result_Display

Module: Student Result

Title: Result information display issue

Steps to Reproduce

1. Log into CUOnline.
2. Open the Result/Grades section.
3. Select the required semester.
4. View the result.

Expected Result

The system should correctly display the student's course names, grades, marks and relevant academic information.

Actual Result

The Result/Grades section displays the student's course names, grades, marks and other available academic information correctly for the selected semester.

Severity

High

Priority

High

Status

Open

7. Defect Report 05

Defect ID: CU-D05

Code/Defect Name: CU_D05_Profile_Validation

Module: Student Profile

Title: Invalid profile information validation

Steps to Reproduce

1. Log into CUOnline.
2. Open the profile section.
3. Enter invalid data into an editable field.
4. Click Save/Update.

Expected Result

The system should validate the entered information and reject invalid data.

Actual Result

The system validates the entered profile information and does not save the invalid value. A validation message is displayed for the incorrect field.

Severity

Medium

Priority

Medium

Status

Open

8. Defect Report 06

Defect ID: CU-D06

Code/Defect Name: CU_D06_Logout

Module: Logout

Title: Session should terminate after logout

Steps to Reproduce

1. Log into CUOnline.
2. Open any authenticated page.
3. Click Logout.
4. Try to access a previously opened authenticated page using the browser's Back button.

Expected Result

The system should require the user to log in again before accessing protected information.

Actual Result

After logout, the previously opened authenticated page cannot be accessed through the browser Back button without logging in again. The system requires authentication to access protected information.

Severity

High

Priority

High
Status
Open

9. CUOnline Defect Summary

Defect ID	Defect Name	Module	Severity	Priority	Status
CU-D01	CU_D01_Login_Validation	Login	Medium	High	Open
CU-D02	CU_D02_Invalid_Login	Login	Medium	High	Open
CU-D03	CU_D03_Duplicate_Course	Course Registration	High	High	Open
CU-D04	CU_D04_Result_Display	Result	High	High	Open
CU-D05	CU_D05_Profile_Validation	Profile	Medium	Medium	Open
CU-D06	CU_D06_Logout	Logout	High	High	Open

10. Overall Conclusion

In **Task 3**, the CUOnline system was considered for defect identification. Different modules such as Login, Course Registration, Result, Profile and Logout were selected for testing. Defects can be documented using defect IDs, reproduction steps, expected results, actual results, severity and priority.